

Raport z Projektu: System OCR dla Tablic Reje- stracyjnych z Wykorzystaniem Metody Dopasowania Wzorców

Data: 21.05.2026

1 Streszczenie

Niniejszy dokument przedstawia wyniki projektu dotyczącego budowy autorskiego systemu optycznego rozpoznawania znaków (OCR) dedykowanego polskim tablicom rejestracyjnym. Rozwiązanie opiera się na bibliotece OpenCV i wykorzystuje metodę dopasowania wzorców (ang. **Template Matching**). W raporcie opisano potok przetwarzania obrazu, w tym ręczne oznaczanie i transformację perspektywiczną, filtrację, binaryzację, segmentację oraz klasyfikację znaków.

2 Wstęp

Zautomatyzowane rozpoznawanie tablic rejestracyjnych (ALPR) jest kluczowym elementem nowoczesnych systemów parkingowych oraz monitoringu ruchu drogowego. Celem projektu było stworzenie lekkiego i wydajnego skryptu w języku Python, zdolnego do odczytywania tekstu ze zdjęć tablic rejestracyjnych, bez konieczności stosowania zasobożernych modeli głębokiego uczenia.

2.1 Założenia Projektowe

W ramach realizacji projektu przyjęto następujące założenia dla przetwarzanych danych wejściowych:

- Analizie poddawane są zdjęcia samochodów, na których widoczne są tablice rejestracyjne. Proces ekstrakcji tablicy oparty jest o ręczne zaznaczenie jej rogów.
- System obsługuje i jest weryfikowany wyłącznie pod kątem polskich tablic rejestracyjnych (wymiary proporcjonalne do 520x114 mm).
- Rozpoznawane obiekty charakteryzują się czarnymi literami umieszczonymi na białym tle.
- Zakłada się występowanie względnie dobrych warunków oświetleniowych na obrazach.
- Celem pobocznym pracy jest ewaluacja autorskiego systemu wizji komputerowej poprzez bezpośrednie porównanie z uniwersalnym, gotowym rozwiązaniem, jakim jest Tesseract OCR.

3 Metodologia

System został podzielony na kilka głównych etapów działania, realizowanych sekwencyjnie. Od ekstrakcji tablicy ze zdjęcia, poprzez jej preprocessing, aż do ostatecznego rozpoznania znaków.

3.1 Transformacja Perspektywiczna (Ekstrakcja Tablicy)

Z uwagi na to, że zdjęcia pojazdów wykonywane są pod różnymi kątami, przed przystąpieniem do analizy konieczne jest „wyprostowanie” tablicy rejestracyjnej. Proces ten zrealizowano metodą transformacji perspektywicznej (ang. **Perspective Transform**) za pomocą funkcji `cv2.warpPerspective`. Operator manualnie zaznacza cztery narożniki tablicy (kolejno: lewy-górny, prawy-górny, prawy-dolny, lewy-dolny) na przeskalowanym w dół podglądzie zdjęcia. Współrzędne są rzutowane na oryginalny, wysokorozdzielczy obraz. Następnie algorytm deformuje wycięty obszar tak, aby wpasować go w idealny prostokąt o standardowych proporcjach polskiej tablicy (520x114 pikseli). Pozwala to wyeliminować zniekształcenia perspektywiczne (skręty, pochylenia), które krytycznie wpływają na skuteczność późniejszego rozpoznawania znaków.

3.2 Preprocessing Obrazu

Aby ułatwić algorytmom segmentacji wykrycie właściwych liter, uzyskany z poprzedniego kroku wycinek tablicy poddawany jest standaryzacji (skrypt `preprocessing.py`). Etap ten składa się z sekwencji klasycznych operacji na obrazie:

1. **Konwersja do skali szarości (cv2.cvtColor):** Odrzucenie informacji o kolorze przyspiesza obliczenia.
2. **Wyrównanie histogramu CLAHE (cv2.createCLAHE):** Zastosowano algorytm **Contrast Limited Adaptive Histogram Equalization**, który adaptacyjnie poprawia kontrast w mniejszych sekcjach obrazu (siatka 8x8). Wzmacnia to widoczność znaków, niwelując lokalne prześwietlenia lub cienie.
3. **Redukcja Szumów (cv2.GaussianBlur):** Aby zapobiec powstawaniu artefaktów podczas binaryzacji (np. z powodu brudu na tablicy), zaaplikowano rozmycie gaussowskie z użyciem jądra wielkości 5x5 pikseli.
4. **Binaryzacja Otsu (cv2.THRESH_OTSU):** Końcowym krokiem jest konwersja obrazu do postaci czarno-białej (1-bitowej). Metoda Otsu automatycznie dobiera optymalny próg odcięcia, oddzielając piksele ciemnego tekstu od jasnego tła.

3.3 Baza Szablonów

Algorytm OCR w pierwszym kroku ładuje referencyjne obrazy poszczególnych liter i cyfr (szablony). Podobnie jak obraz wejściowy, są one poddawane binaryzacji, aby zminimalizować różnice i umożliwić ustandaryzowane porównanie morfologii znaków.

3.4 Segmentacja Znaków

Przed przystąpieniem do segmentacji, na przetworzonym, binarnym wycinku tablicy przeprowadzana jest inwersja kolorów (`cv2.bitwise_not`). Algorytmy detekcji konturów w OpenCV (`cv2.RETR_EXTERNAL`) domyślnie poszukują jasnych obiektów na ciemnym tle, dlatego konieczna jest zamiana czarnych znaków na białe.

Z uwagi na obecność szumów, takich jak ramki tablicy, śruby montażowe czy naklejki legalizacyjne, zastosowano rygorystyczne filtry geometryczne:

- **Pole powierzchni (Area):** Odrzucane są obiekty zbyt małe (np. drobne artefakty) oraz zbyt duże.
- **Proporcje (Aspect Ratio):** Wykorzystano fakt, że polskie znaki rejestracyjne są wysokie i wąskie. Skrypt akceptuje wyłącznie obszary, których stosunek szerokości do wysokości mieści się w przedziale od 0.2 do 0.9.

Wyselekcjonowane w ten sposób obwiednie są następnie sortowane rosnąco według współrzędnej X, co zapewnia odczyt symboli od lewej do prawej.

3.5 Klasyfikacja metodą Template Matching

Każdy wysegmentowany znak jest wycinany i skalowany do ujednoliconego wymiaru 40×80 pikseli. Jest to wymóg konieczny dla metody **Template Matching**, opierającej się na nakładaniu wzorców na badany fragment.

System systematycznie porównuje dany wycinek ze wszystkimi wgranymi szablonami, korzystając ze znormalizowanego współczynnika korelacji (`cv2.TM_CCORF_NORMED`). Metoda ocenia nakładanie się pikseli po nałożeniu na siebie obrazów, dając wynik z przedziału $[-1, 1]$. Jeżeli dla danego znaku korelacja z określonym szablonem jest najwyższa i przekracza 50%, litera lub cyfra zostaje dodana do zwracanego łańcucha tekstowego.

4 Wyniki

Testy systemu wykazały zadowalającą skuteczność dla czystych obrazów, dla których prawidłowo wyznaczono rogi transformacji perspektywicznej. Zastosowanie filtrów proporcji (0.2 - 0.9) skutecznie eliminuje problem błędnego interpretowania naklejek i znaków interpunkcyjnych.

Zidentyfikowano jednak pewne ograniczenia:

- **Jakość manualnej ekstrakcji:** Skuteczność dopasowania drastycznie spada, jeśli operator nierówno oznaczy narożniki, co doprowadza do zachwiania perspektywy i nienaturalnego wydłużenia lub spłaszczenia liter, zaburzając działanie filtra „Aspect Ratio” oraz finalnej korelacji ze sztywnymi szablonami.
- **Zależność od czcionki:** Metoda **Template Matching** wymaga, aby szablony ściśle odpowiadały krojowi pisma na zdjęciu. W przypadku zabrudzeń psujących kształt obwiedni, algorytm gubi znaki.

5 Porównanie z Tesseract OCR

(Sekcja w przygotowaniu - oczekiwanie na kompletne dane eksperymentalne) Docelowo w tym rozdziale zestawione zostaną osiągi skuteczności i elastyczności autorskiego systemu bazującego na OpenCV względem silnika Tesseract OCR. Analiza uwzględni dokładność odczytu ciągów znaków oraz czas analizy jednego obrazka, bazując na wcześniej zdefiniowanych założeniach.

6 Wnioski

Zaproponowane rozwiązanie, rozbudowane o moduł transformacji perspektywicznej i adaptacyjnego ulepszania obrazu (CLAHE), stanowi solidną i wydajną implementację klasycznego potoku widzenia komputerowego. Najslabszym ogniwem sytemu w obecnej iteracji jest wymóg manualnego oznaczania narożników.

W przyszłym rozwoju projektu, obok eliminacji ręcznej ingerencji (np. detekcja tablic modelem YOLO), zaleca się rozważenie integracji z lekkimi sieciami neuronowymi (np. kNN lub SVM) ekstrahującymi niezależne od skali cechy wektorowe (HOG), co znacząco podniosłoby odporność modułu klasyfikacji na błędy perspektywy, których nie potrafi skompensować dopasowanie wzorców.